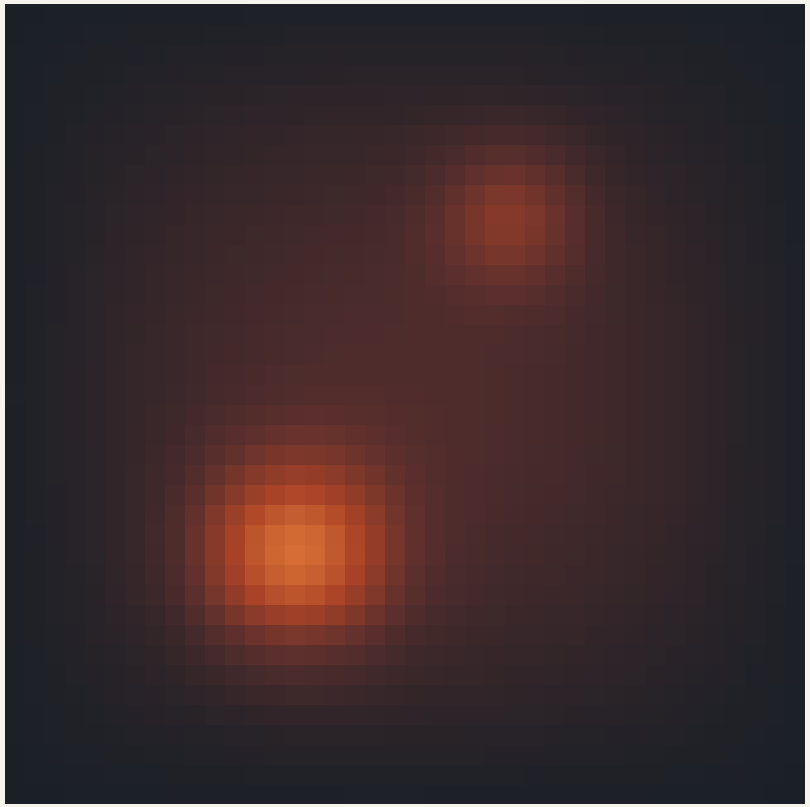
Heat diffusion, in parallel

We simulate a cooling plate and compare two ways to solve the same temperature equations: **Jacobi** and **conjugate gradient (CG)**.

> Research question
> How much faster do more processes make the solve, and when does MPI communication become the main cost?



Brighter cells are warmer. This separate visualization uses a 128 × 128 grid after five time steps.

Srikar Palani and Suprathik Vinayakula
CS598 Midterm Project · Fall 2026

The study compares time to the same residual tolerance, across 1–64 MPI processes and 280 measured runs.

Each time step requires a linear solve

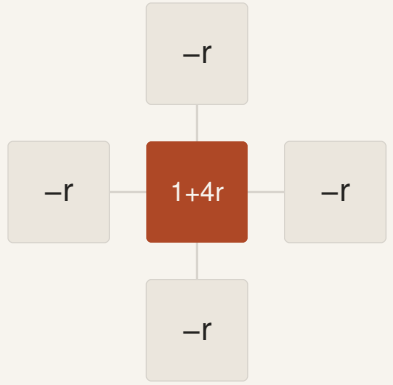
Physical model: a unit-square plate, warm regions inside, and edges held at zero temperature.

The temperature model

An **N × N grid** represents the interior. Heat moves between neighbors; larger N gives a finer grid. Backward Euler advances one physical step by solving:

$$A\,x = b$$

b is the current temperature field.
x is the next temperature field.
A describes the neighbor interactions.

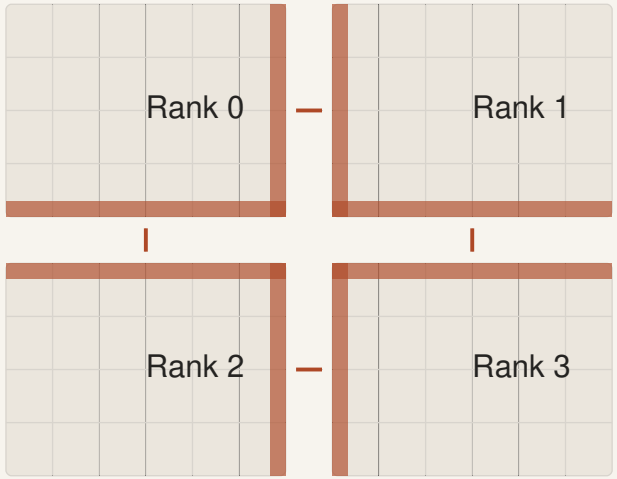


Each point interacts with four neighbors. The code calculates these five-point interactions directly instead of storing a huge matrix.

A physical time step advances the simulation. Solver iterations are the repeated updates needed to solve A x = b within that step.

MPI processes share the plate

P is the number of MPI processes. A process is also called a rank; each rank owns one grid block.



Four-rank example. Orange strips indicate exchanged border values, called halos.

Two communication patterns

Neighbors exchange halos

A rank needs values just beyond its block to update points along the edge.

All ranks combine scalars

Global reductions combine local results for dot products and convergence checks. These calls can include waiting for other ranks.

More ranks reduce the cells owned by each process, but halo exchanges and global reductions remain necessary.

Two solvers, one accuracy target

Each physical step solves the same A x = b, starting from x = b. Only the iterative method changes.

Jacobi

Repeatedly corrects each temperature estimate using the current residual. Each update is simple, but many may be needed.

- Neighbor exchange every update
- Convergence check after update 1, every 10 updates, and at the iteration cap

Conjugate gradient (CG)

Builds search directions that usually make more progress per update. Each update costs more because it needs additional calculations and global communication.

- Neighbor exchange every update
- Two global dot products per update

Residual: how well the computed x satisfies A x = b, relative to the size of b. Both methods must pass an independently recomputed residual ≤ 10^{-8}.

We compare complete time to solution. A cheap iteration is useful only if the method does not need too many of them.

What the 56 experiment groups contain

A configuration chooses a solver, grid size N, process count P, and timestep parameter r.

Study	Settings	Groups
Strong scaling	2 solvers × N = 512, 1024 × 9 process counts P = 1, 2, 4, 8, 16, 24, 32, 48, 64; r = 2	36
Weak scaling	2 solvers × (N, P) = (256, 1), (512, 4), (1024, 16), (2048, 64); r = 2	8
Timestep sensitivity	2 solvers × N = 512, 1024 × r = 0.5, 2, 8 P = 16	12

56 groups × 5 trials = 280 runs

Every run measures five physical time steps. Some numerical settings repeat across studies; they remain separate experiment groups.

The timestep parameter r sets the size of a physical step relative to grid spacing; its effect is explained on slides 15–16.

How the timings were collected

Two exclusive AMD EPYC 7713 nodes on the Illinois Campus Cluster, with ranks bound to physical cores.

One measured run

Discard a warm-up, reset the temperature field, then measure five physical steps. Runtime is the sum of each step's slowest-rank time.

All ranks must finish. Each step's compute and MPI breakdown comes from that same selected rank.

A fair reading of the results

We report the median of five trials. Whiskers show the observed min–max range, not confidence intervals.

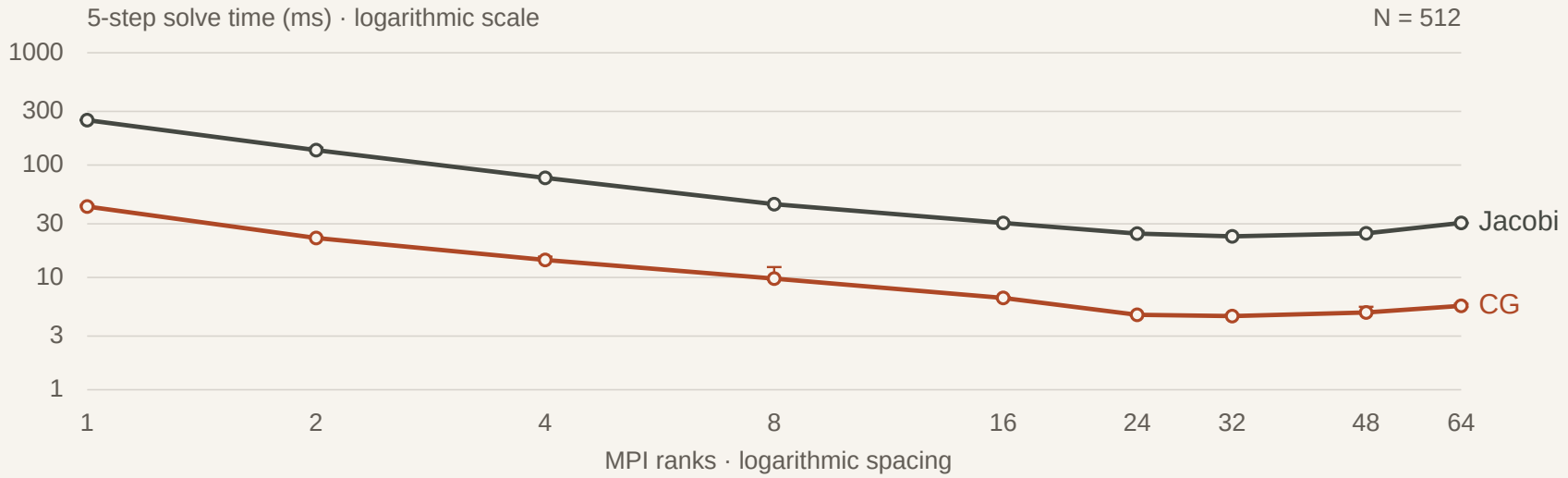
Timing includes solver calculations, checks, and MPI calls. Field initialization, final accuracy verification, and output are excluded.

Placement matters: P = 1 uses one node. P ≥ 2 spreads the ranks evenly over both nodes. Five nearby trials share the same allocation.

All plotted runtime values are milliseconds for the entire five-step solve, rather than a single iteration or a single physical step.

Strong scaling on a 512 × 512 grid

Strong scaling fixes the total grid and adds ranks. Ideal behavior: twice as many ranks would halve runtime. Here r = 2.



CG is faster at every sampled P

At P = 32: CG takes 4.525 ms; Jacobi takes 23.190 ms. That is a 5.13-fold runtime advantage.

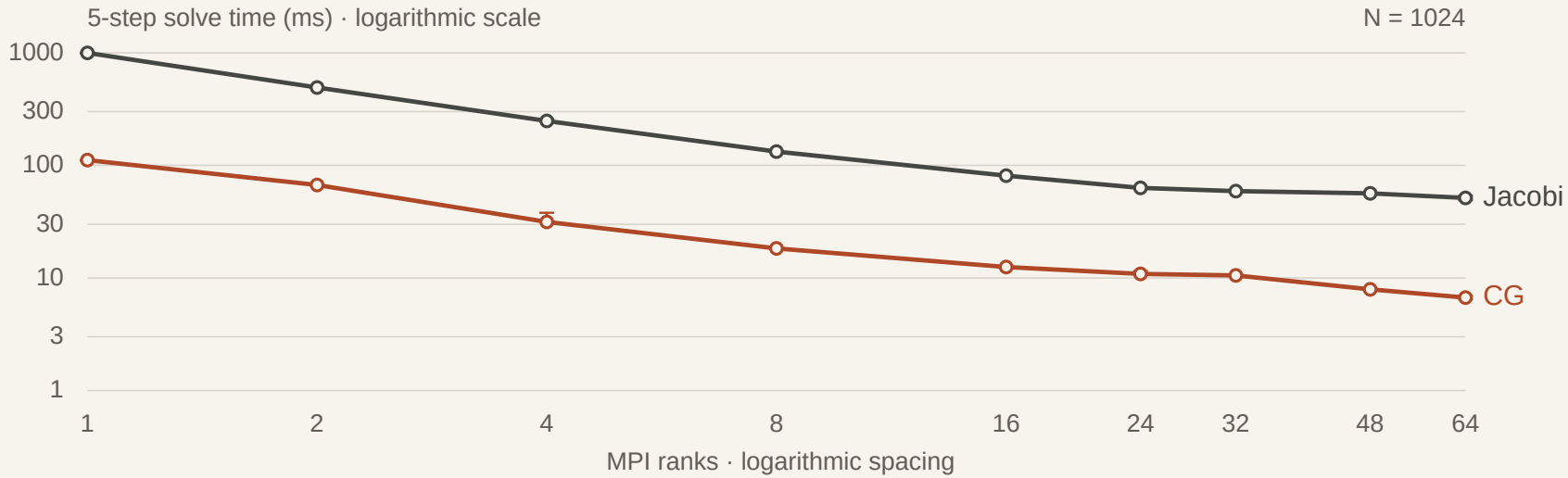
More ranks eventually stop helping

Both methods reach their best sampled runtime at 32 ranks. Runtime increases at 48 and 64.

Read downward as faster. Both axes use logarithmic spacing; points are five-trial medians, with min–max whiskers.

A larger grid benefits through 64 ranks

Strong scaling with N = 1024, r = 2, and the same solvers, accuracy target, hardware, and five-step timing.



CG retains the lower runtime

At P = 64: CG takes 6.689 ms; Jacobi takes 51.396 ms. CG is 7.68 times faster.

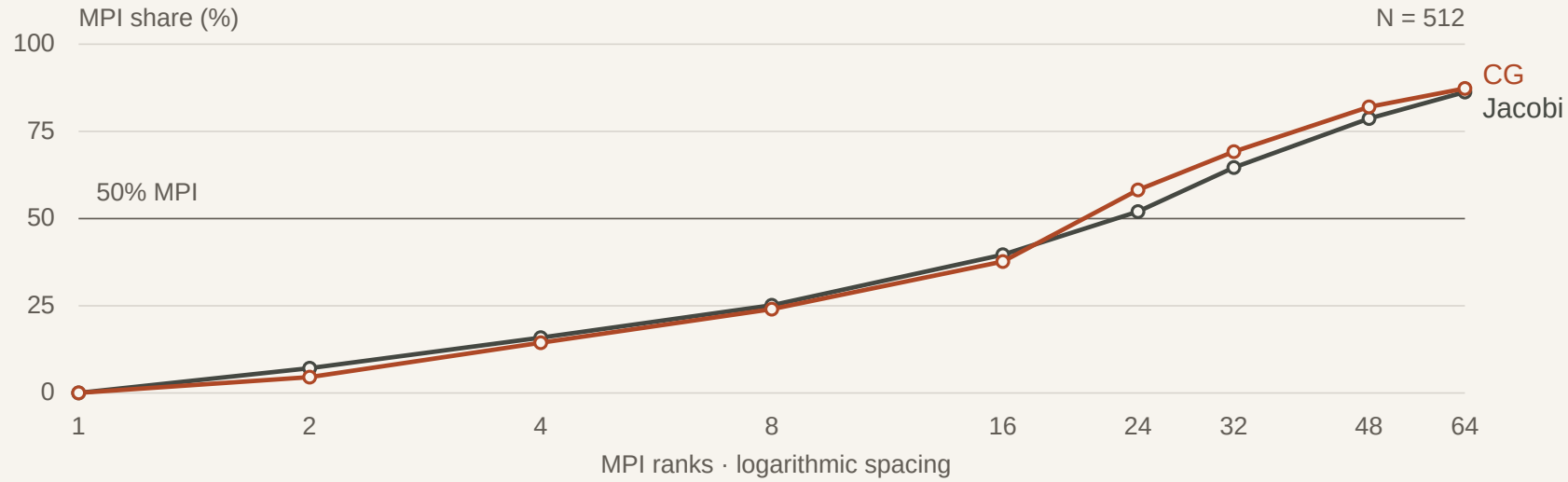
The larger workload scales farther

Both methods still improve at 64 ranks. Each rank retains more useful work than on the 512 × 512 grid.

Speedup is T_1 / T_p for the same solver. A higher relative speedup does not necessarily mean a lower absolute runtime.

MPI dominates sooner on the smaller grid

N = 512, r = 2. MPI share = (halo + reduction time) / (local compute + halo + reduction time).



The crossover is between 16 and 24 ranks

Both solvers move from below 50% MPI time at P = 16 to above 50% at P = 24.

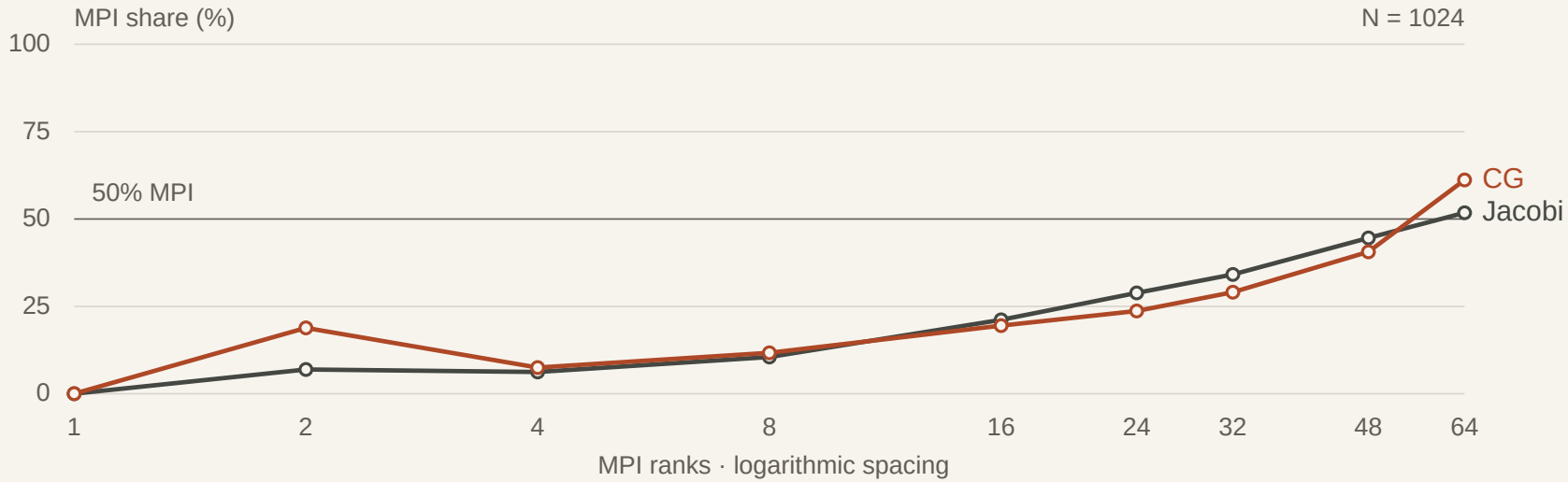
Why the share grows

Each rank owns fewer cells as P rises, while neighbor exchanges and global reductions remain necessary.

The dashed line marks a measured 50% share. MPI time includes waiting and imbalance; other bookkeeping is excluded.

Larger grids postpone MPI dominance

N = 1024, r = 2. The same MPI-share definition and 50% threshold apply to both solvers.



The crossover moves to 48–64 ranks

At P = 64, the MPI shares reach 51.8% for Jacobi and 61.2% for CG.

A high MPI share can still accompany gains

Both runtimes continue falling through 64 ranks, even though MPI becomes the larger measured component.

A percentage describes the balance of time, not its absolute size. Compare it with the runtime plot before choosing P.

CG wins by needing far fewer updates

Same problem: N = 1024, P = 64, r = 2, tolerance 10^{-8}. Counts and times cover all five physical steps.

Measured quantity	Jacobi	CG
Total solver updates	450	24
Median five-step runtime	51.396 ms	6.689 ms
Runtime divided by updates	0.114 ms/update	0.279 ms/update

One CG update costs more

It performs extra vector work and global dot products. The per-update figure also includes solver setup and checks.

Its smaller update count wins overall

Jacobi performs 18.75 times as many updates. CG’s fewer updates outweigh its higher cost per update.

These are solver iterations within five simulation steps. They do not mean that the two methods simulated different physical durations.

Weak scaling keeps cells per rank fixed

Weak scaling asks whether more processes can handle a proportionally larger problem in a similar amount of time.

MPI ranks P	Total interior grid	Owned cells per rank
1	256 × 256	256 × 256
4	512 × 512	256 × 256
16	1024 × 1024	256 × 256
64	2048 × 2048	256 × 256

Strong scaling

The total problem stays fixed. More processes should reduce runtime.

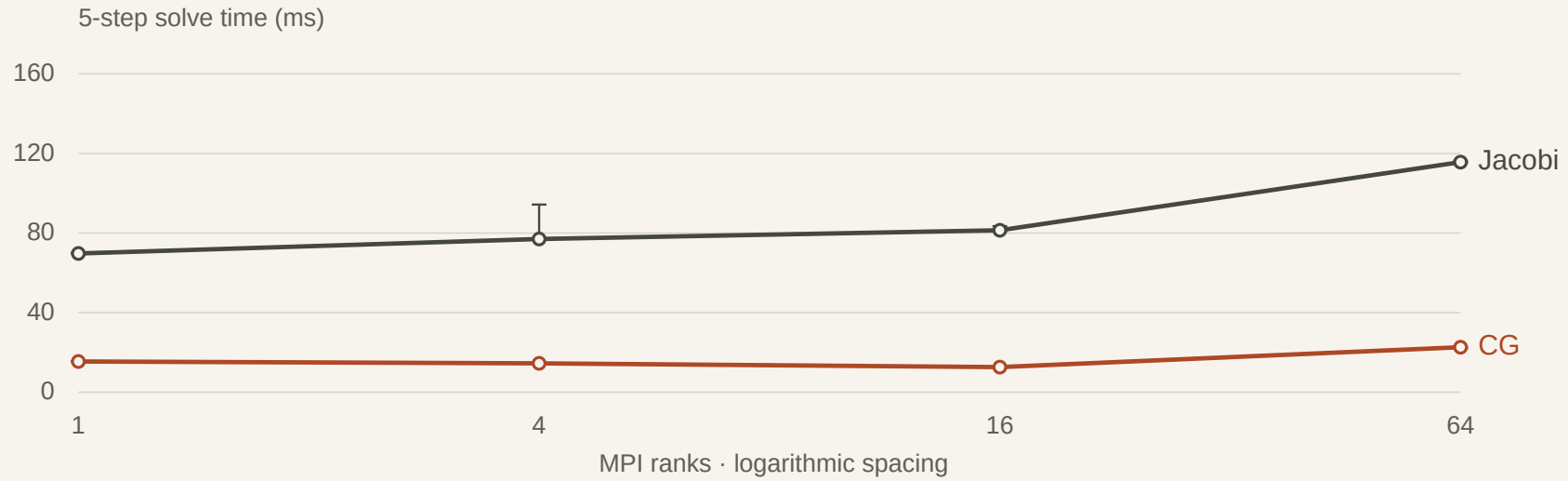
Weak scaling

The problem grows with P. Ideal runtime stays constant if work per rank and iteration counts stay constant.

These weak-scaling runs use r = 2, the same two solvers and tolerance, and five physical steps per run.

Weak-scaling runtime increases overall

Total grid grows from 256 × 256 at P = 1 to 2048 × 2048 at P = 64. Each rank still owns 256 × 256 cells.



Jacobi: 69.7 to 115.7 ms

Its total five-step runtime rises as the overall problem and process count grow.

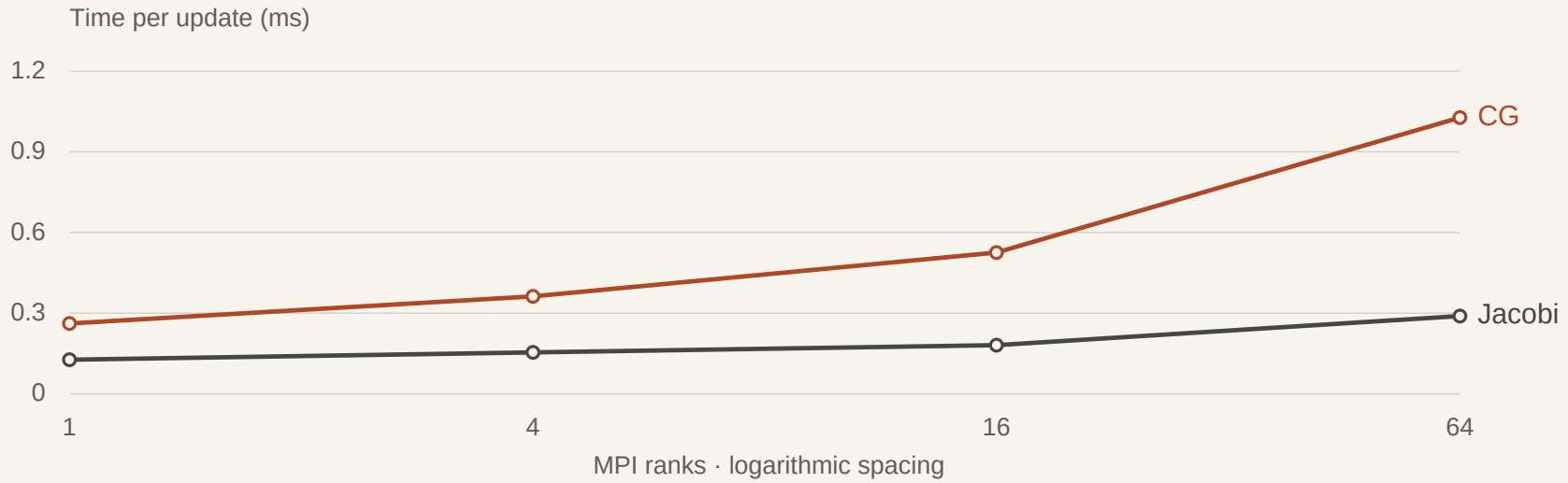
CG: 15.4 to 22.6 ms

CG remains faster. Its intermediate runtimes fall before the increase at 64 ranks.

A perfectly flat curve would indicate constant runtime. Here, changing iteration counts also affect the curve; the next slide separates that effect.

Per-update time exposes hidden overhead

Same weak-scaling runs. Time per update = total measured solve time divided by total solver updates.



The cost per update rises

Jacobi: 0.127 to 0.289 ms. CG: 0.262 to 1.027 ms, from 1 to 64 ranks.

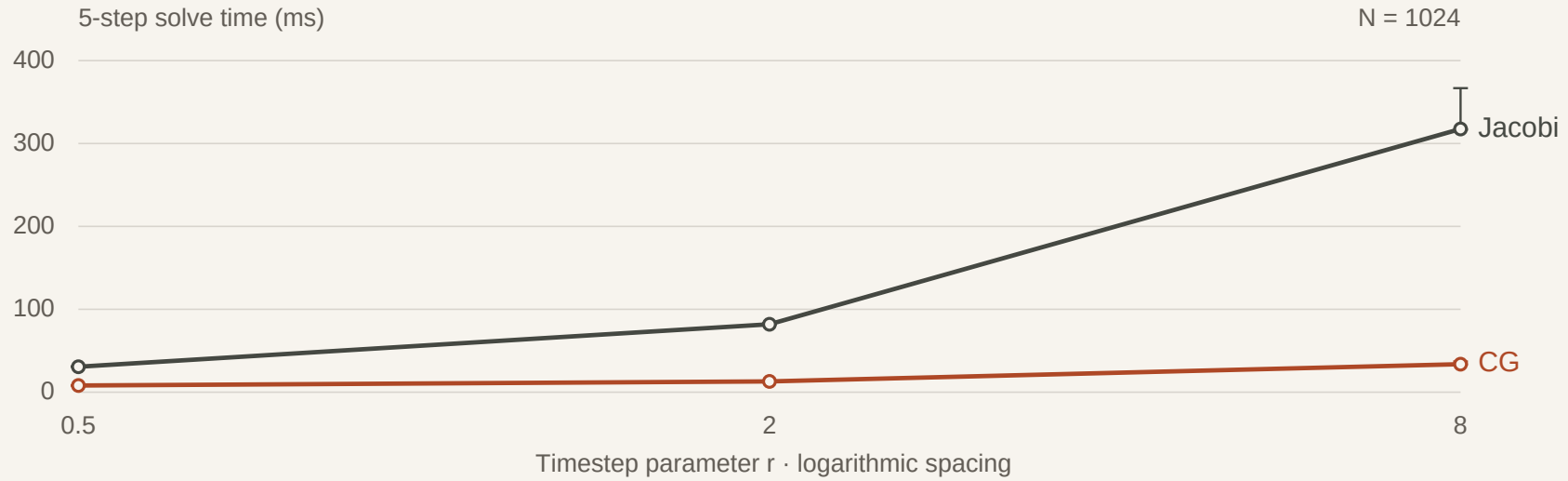
Fewer updates partly mask that rise

Jacobi update counts fall from 550 to 400; CG counts fall from 59 to 22 across the five steps.

This ratio includes setup and convergence checks within the solve. It is not an isolated update-kernel timer or a pure network measurement.

Larger timesteps make each solve harder

At N = 1024 and P = 16, vary r = αΔt / h². Here α = 1 is diffusivity, Δt is timestep, and h is grid spacing.



r = 0.5 to 8 increases runtime

Jacobi: 30.7 to 317.6 ms. CG: 8.06 to 33.8 ms. Each measurement still covers five steps.

What changing r means

With the grid fixed, larger r advances farther in physical time per step and changes the system A x = b.

These runs simulate different physical durations. The plot compares solve workloads, not the cost of reaching the same final physical time.

More iterations explain the extra time

Timestep sensitivity at N = 1024, P = 16. All values below are totals or medians over the same five-step workload.

Timestep r	Jacobi updates	CG updates	Jacobi time	CG time
0.5	150	11	30.675 ms	8.060 ms
2	450	24	81.935 ms	12.983 ms
8	1920	85	317.589 ms	33.782 ms

The numerical system changes

A larger r needs more solver updates for this initial field and tolerance. CG still needs far fewer updates than Jacobi.

Stability is only one consideration

Backward Euler is stable for large timesteps, but a stable step can still be expensive or less accurate.

A larger timestep is not automatically the cheaper or more accurate choice. Equal step counts do not imply equal simulated durations.

Correctness checks and interpretation limits

Every benchmark passed the same independently recomputed relative-residual tolerance of 10^{-8}.

Why trust the computed solutions?

Tests match an independent serial solver and analytic sine solution. They also cover refinement, halos, physical boundaries, uneven partitions, and failure cases.

All 280 runs passed. The maximum residual was **9.985×10^{-9}**. Diagnostics checked timing selection and rejected a CG result with a misleading internal residual.

What the measurements cannot establish

MPI time includes waiting and imbalance. P = 1 uses one node while larger P uses two, so topology and rank count change together.

Trials share one allocation; they do not capture day-to-day variation. Fixed r changes the physical timestep with N. Results depend on this initial field, tolerance, hardware, and implementation.

Residual tolerance checks the linear solve. It does not by itself guarantee that different grids and timesteps have the same physical accuracy.

What the study establishes

Two solvers, the same implicit heat model, and 280 validated measurements on a two-node allocation.

01 **CG lowers time to solution**

It wins every tested strong-scaling comparison because far fewer updates offset its higher cost per update.

02 **Grid size changes useful parallelism**

The 512 × 512 grid is fastest at 32 sampled ranks. The 1024 × 1024 grid still improves at 64.

03 **Read runtime and MPI share together**

MPI can exceed half the measured time while additional ranks still reduce total runtime.

Evidence and reproduction

The repository contains the solver, automated tests, Slurm scripts, raw JSON records, and analysis.

Build with CMake and OpenMPI. Run MPI work in an allocated compute-node job.

Build and run instructions

Reader-focused project report

Measured summary CSV

These findings apply to this workload; other hardware, initial fields, tolerances, or solvers can change the outcome.